Para aplicar o **encapsulamento**, precisamos "esconder" os dados dentro da classe para que eles não possam ser alterados de forma errada ou direta por quem está de fora.

Em Python, fazemos isso colocando dois sublinhados (`__`) antes do nome do atributo. Isso torna a variável **privada**. Depois, criamos os métodos seguros para ler (`@property`) e, se necessário, alterar esses valores (os `setters`).

Aqui está o script atualizado aplicando o encapsulamento, com comentários explicativos para os alunos:

Python

```python
class Terreno:
    def __init__(self, largura, comprimento, valor_metro_quadrado):
        # Os dois sublinhados (__) tornam os atributos PRIVADOS (Encapsulamento).
        # Ninguém de fora da classe pode acessar ou alterar "meu_terreno.__largura" di
        self.__largura = largura
        self.__comprimento = comprimento
        self.__valor_metro_quadrado = valor_metro_quadrado

    # ==========================================
    # GETTERS (Métodos para LER os dados seguros)
    # ==========================================
    @property
    def largura(self):
        return self.__largura

    @property
    def comprimento(self):
        return self.__comprimento

    @property
    def valor_metro_quadrado(self):
        return self.__valor_metro_quadrado

    # ==========================================
    # SETTERS (Métodos para ALTERAR os dados com regras)
    # ==========================================
    @largura.setter
    def largura(self, novo_valor):
        if novo_valor > 0:
            self.__largura = novo_valor
        else:
            print("Erro: A largura deve ser maior que zero!")
```

```
    @comprimento.setter
    def comprimento(self, novo_valor):
        if novo_valor > 0:
            self.__comprimento = novo_valor
        else:
            print("Erro: O comprimento deve ser maior que zero!")

    # ========================================
    # MÉTODOS DE AÇÃO (Regras de negócio)
    # ========================================
    def calcular_area(self):
        # Como estamos DENTRO da classe, podemos acessar os atributos privados com __
        return self.__largura * self.__comprimento

    def calcular_preco(self):
        area = self.calcular_area()
        return area * self.__valor_metro_quadrado

    def mostrar_resultados(self):
        print(f"Area do terreno = {self.calcular_area():.2f}")
        print(f"Preco do terreno = {self.calcular_preco():.2f}\n")


# ========================================
# FLUXO PRINCIPAL DO PROGRAMA
# ========================================

largura_digitada = float(input("Digite a largura do terreno: "))
comprimento_digitado = float(input("Digite o comprimento do terreno: "))
valor_digitado = float(input("Digite o valor do metro quadrado: "))

# Instanciando o objeto
meu_terreno = Terreno(largura_digitada, comprimento_digitado, valor_digitado)

# O usuário não percebe a diferença, mas se ele tentar fazer isso:
# meu_terreno.largura = -50
# O "setter" vai bloquear, protegendo a integridade do terreno!

meu_terreno.mostrar_resultados()
```

O que os alunos aprendem com esta versão:

Atributos Privados (__): Entendem que os dados sensíveis ficam trancados dentro de um cofre.

Segurança (Setters): Se alguém tentar colocar uma largura negativa por engano, a classe recusa e avisa do erro, algo impossível de fazer quando as

variáveis ficam soltas.

Leitura Elegante (`@property`): Demonstra que, mesmo o dado estando trancado, podemos criar uma "janela" para que o fluxo principal veja a largura e o comprimento de forma limpa.

> explique melhor o que é e como fazer o encapsularmento.
> pois não entendi.

É super normal não entender de primeira! O encapsulamento é um dos conceitos mais importantes da Orientação a Objetos, e exige uma pequena mudança na forma como pensamos a programação.

Para explicar de forma bem simples para os seus alunos, vamos esquecer o código por um minuto e pensar no mundo real.

1. O que é Encapsular? (A Analogia da Televisão)

Pense na palavra **Cápsula**. Quando você toma um remédio em cápsula, o pó não fica solto, ele fica protegido lá dentro.

Agora imagine uma **Televisão**.
Dentro da TV existem fios, placas de circuito e energia em alta tensão. Se você enfiar a mão lá dentro para tentar aumentar o volume puxando um fio, você pode quebrar a TV ou tomar um choque.

A TV **esconde (encapsula)** a parte perigosa/complexa dentro da carcaça de plástico.

Como você interage com ela, então? Através do **Controle Remoto**. Os botões do controle são a forma segura que o fabricante criou para você mexer na TV. Se você apertar o botão de diminuir o volume até o zero, a TV não deixa o volume ficar negativo (tipo -5), porque existe uma regra interna protegendo isso.

Na programação é a mesma coisa:

As variáveis (atributos) são os fios da TV. Nós não queremos que ninguém mexa nelas diretamente.

Os métodos são os botões do controle remoto. Eles permitem que o usuário veja ou altere os dados, mas seguindo as regras que nós definimos.

2. Por que isso é útil? (O Problema)

Imagine um sistema de Conta Bancária onde o saldo não está encapsulado (não está protegido):

```python
minha_conta = ContaBancaria(100)

# Um programador descuidado faz isso:
minha_conta.saldo = -50000
```

Pronto, o sistema quebrou! Alguém alterou o saldo diretamente para um valor absurdo, porque a variável estava solta e desprotegida.

3. Como fazer o Encapsulamento em Python (Passo a Passo)

Para encapsular, fazemos duas coisas: **Trancamos o cofre** e **Criamos uma porta segura**.

Passo 1: Trancar o cofre (Tornar Privado)

Em Python, para avisar que uma variável é secreta e não deve ser mexida de fora, nós colocamos **dois sublinhados (__)** antes do nome dela.

```python
class ContaBancaria:
    def __init__(self, saldo_inicial):
        # O __saldo agora está protegido! Ninguém de fora consegue mexer diretamente.
        self.__saldo = saldo_inicial
```

Passo 2: Criar a "Porta Segura" (Getters e Setters)

Se trancarmos o saldo, como o usuário vai saber quanto dinheiro ele tem? Ou como vai depositar? Criando métodos específicos (os botões do controle remoto).

Exemplo usando métodos normais (ótimo para iniciar os alunos):

Python

```python
class ContaBancaria:
    def __init__(self, saldo_inicial):
        self.__saldo = saldo_inicial

    # BOTÃO DE VER (Getter): Apenas mostra o valor, não deixa alterar.
    def ver_saldo(self):
        return self.__saldo

    # BOTÃO DE ALTERAR (Setter): Cria regras antes de mudar o valor real.
    def depositar(self, valor):
        if valor > 0:
            self.__saldo = self.__saldo + valor
        else:
            print("Erro: Você não pode depositar um valor negativo!")
```

Veja como fica seguro usar a classe agora:

Python

```python
conta = ContaBancaria(100)

# Para ver o saldo, o usuário é obrigado a usar o método:
print(conta.ver_saldo())

# Para colocar dinheiro, ele é obrigado a usar o método, que vai passar pelo 'if':
conta.depositar(-50) # O programa não quebra, ele imprime a mensagem de Erro!
```